SC 677 703 · Final Presentation · Script สำหรับผู้นำเสนอ

Early Detection of Multimodal Fake News via Reinforced Propagation Path Generation

L. Zhang et al. · IEEE TKDE 37(2), 2025 · สไลด์: MS_DPMLA_Final Presentation.pdf (42 หน้า)

เวลารวมโดยประมาณ

32:50 นาที

วิธีใช้

อ่านตัวหนาให้ชัด · กล่องสีฟ้า = สิ่งที่ควรชี้หรือทำบนสไลด์ · เวลาเป็นค่าประมาณ ถ้าเวลาไม่พอ ย่อสไลด์ 11–17 (รายละเอียด Module 1–2) และ 24–25 ได้ ประหยัดราว 6 นาที

เวลาแยกตามหัวข้อ

หัวข้อ	สไลด์	เวลา
Prologue	1	0:30
Introduction	2–6	3:45
Methodology	7–21	12:50
Data	22–25	3:05
Experiment	26–29	2:30
Result	30–32	2:35
Ablation Study	33	0:55
Conclusion	34	0:45
Limitation & Critique	35–41	5:40
Closing	42	0:15
รวม	1–42	**32:50**

ประเด็นหลักที่ต้องสื่อให้ได้

- ปัญหา: ตรวจจากเนื้อหาเร็วแต่ไม่แม่น ตรวจจากเส้นทางแม่นแต่ช้า → paper สร้างเส้นทางในอนาคตด้วย RL
- วิธี: Phase I (Contrastive Learning → User Embedding → RL สร้าง G′) และ Phase II (Temporal-aware Detection)
- ผลของเรา: PHEME / GossipCop ตรง paper · PolitiFact ต่ำกว่า 7.6 pp · ไม่ใช้ G′ ได้เท่ากันหรือดีกว่า
- วิพากษ์: code ใช้ไม่ได้ · ค่าพารามิเตอร์ไม่ครบ · ข้อมูลไม่ครบ · ข้อจำกัดเครื่องของเรา

Prologue · 0:30 นาที



1 หน้าปก

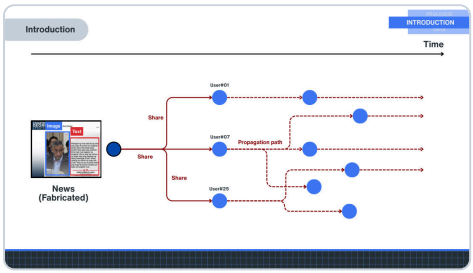
~30 วินาที

สวัสดีครับอาจารย์และเพื่อน ๆ ทุกคน วันนี้ผมจะนำเสนอ paper เรื่อง **Early Detection of Multimodal Fake News via Reinforced Propagation Path Generation** หรือเรียกสั้น ๆ ว่า **RPPG-Fake** ของ L. Zhang และคณะ ตีพิมพ์ใน IEEE Transactions on Knowledge and Data Engineering ปี 2025

นอกจากอธิบาย paper แล้ว ผมได้ลองนำ code มารันจริงด้วย ซึ่งจะเล่าผลและข้อจำกัดที่พบในช่วงท้ายครับ

- ยิ้ม แนะนำตัว แล้วเปลี่ยนหน้า

Introduction · 3:45 นาที



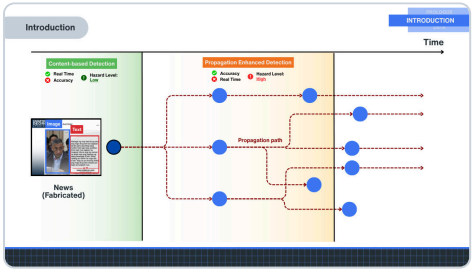
2 Introduction · ข่าวปลอมแพร่อย่างไร

~45 วินาที

เริ่มจากภาพนี้ก่อนครับ ทางซ้ายคือข่าวปลอมหนึ่งข่าว ซึ่งมีทั้ง **ข้อความ (Text)** และ **รูปภาพ (Image)** เมื่อข่าวถูกโพสต์ ผู้ใช้คนแรก ๆ เช่น User#01, #07, #25 จะเริ่มแชร์

จากนั้นคนอื่นก็แชร์ต่อจากคนเหล่านี้ไปเรื่อย ๆ ตามแกนเวลา เส้นทางที่ข่าวเดินทางผ่านผู้ใช้นี้เรียกว่า **Propagation path** หรือเส้นทางการแพร่กระจาย ซึ่งเป็นข้อมูลสำคัญที่ paper นี้ใช้

- ชี้ข่าว → ผู้ใช้ 3 คนแรก → เส้นประที่ขยายออก



3 Introduction · 2 แนวทางเดิม

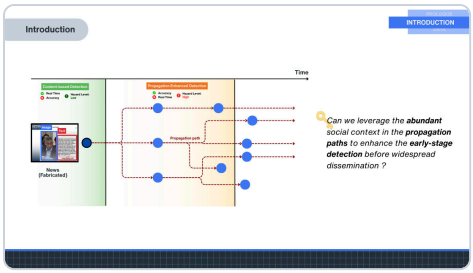
~60 วินาที

งานตรวจจับข่าวปลอมที่ผ่านมาแบ่งได้ 2 แนวทาง

แนวทางแรกคือ **Content-based Detection** ดูแค่เนื้อหาของข่าว ข้อดีคือตรวจได้ทันทีแบบ real time และความเสียหายต่ำ เพราะยังไม่มีใครแชร์ แต่ความแม่นยำไม่สูง เพราะข่าวปลอมสามารถเขียนเลียนแบบข่าวจริงได้

แนวทางที่สองคือ **Propagation Enhanced Detection** ใช้เส้นทางการแพร่ด้วย ทำให้แม่นยำกว่า แต่ต้องรอให้ข่าวแพร่ไปก่อน กว่าจะตรวจได้คนก็เห็นข่าวไปมากแล้ว paper เรียกว่า **social hazard สูง**

- ชี้กล่องเขียว (เร็วแต่ไม่แม่น) แล้วกล่องส้ม (แม่นแต่ช้า)



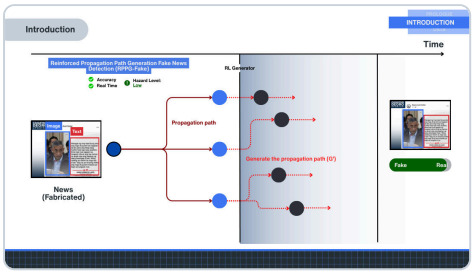
4 Introduction · คำถามวิจัย

~30 วินาที

จึงเกิดเป็นคำถามวิจัยของ paper นี้ครับ **เราจะใช้ข้อมูลทางสังคมจากเส้นทางการแพร่มาช่วยตรวจจับตั้งแต่ช่วงแรก ก่อนที่ข่าวจะแพร่ไปมาก ได้หรือไม่**

คือต้องการทั้งความเร็วของแนวทางแรก และความแม่นยำของแนวทางที่สอง

▸ เว้นจังหวะหลังอ่านคำถาม



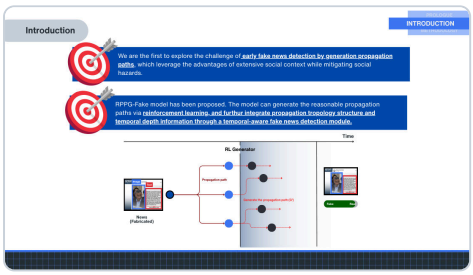
5 Introduction · แนวคิด RPPG-Fake

~50 วินาที

คำตอบของ paper คือ **RPPG-Fake** แทนที่จะรอเส้นทางจริง ให้ใช้ **RL Generator** “สร้าง” เส้นทางการแพร่ในอนาคตขึ้นมาเอง เรียกว่า **G′** โดยดูจากเนื้อหาข่าวและผู้ใช้ 3–5 คนแรกที่แชร์จริง

แล้วนำเส้นทางที่สร้างขึ้นนี้ไปช่วยตัดสินว่าข่าวเป็น Fake หรือ Real ผลคือได้ทั้งความแม่นยำ ความเร็ว และความเสียหายต่ำ

▸ ชี้จุดสีเทา = ผู้ใช้ที่ model สร้างขึ้นเอง



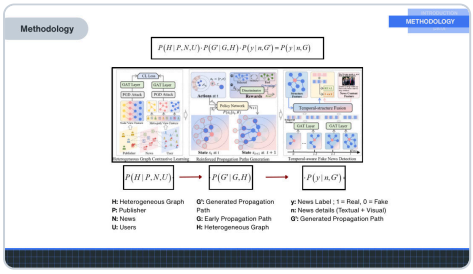
6 Introduction · Contribution

~40 วินาที

paper อ้าง contribution หลัก 2 ข้อ ข้อแรก เป็นงานแรกที่ตรวจจับข่าวปลอมช่วงแรกด้วยการ **สร้างเส้นทางการแพร่** ได้ประโยชน์จากข้อมูลสังคมโดยลดความเสียหาย

ข้อสอง เสนอ model RPPG-Fake ที่สร้างเส้นทางด้วย **Reinforcement Learning** และรวมทั้งโครงสร้างของเส้นทางและข้อมูลเชิงเวลาด้วย **Temporal-aware Detection**

Methodology · 12:50 นาที



7 Methodology · ภาพรวมเชิงความน่าจะเป็น

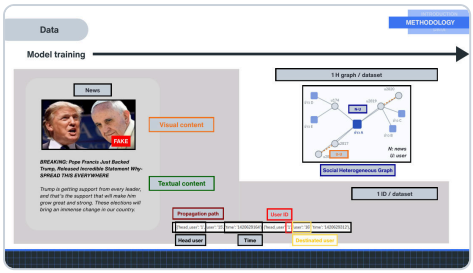
~60 วินาที

ในเชิงคณิตศาสตร์ paper แยกปัญหาออกเป็น 3 ส่วนต่อกัน

ส่วนแรก **P(H | P, N, U)** สร้าง Heterogeneous Graph H จาก Publisher, News และ User ส่วนที่สอง **P(G′ | G, H)** สร้างเส้นทาง G′ จากเส้นทางช่วงแรก G และกราฟ H ส่วนสุดท้าย **P(y | n, G′)** ทำนาย label y จากเนื้อหาข่าว n ซึ่งมีทั้งข้อความและภาพ ร่วมกับ G′

ทั้งสามส่วนตรงกับ 3 module ในภาพด้านบน ซึ่งผมจะอธิบายทีละ module ครับ

▸ หมายเหตุ: บนสไลด์เขียน y: 1 = Real, 2 = Fake แต่ใน paper คือ 1 = Real, 0 = Fake



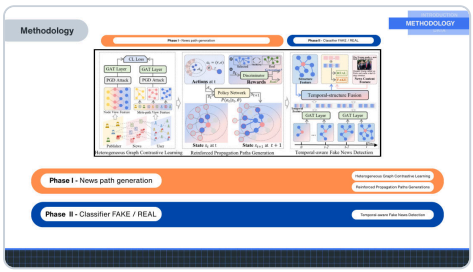
8 Data · input ที่ model ใช้ (ก่อนเข้า 2 Phase)

~40 วินาที

ก่อนลงรายละเอียด ดูก่อนว่า model ใช้ข้อมูลอะไร ข่าวหนึ่งข่าวมี **ภาพ**, **ข้อความ** และ **เส้นทางการแพร่** ที่เก็บเป็น head user, user ที่แชร์ และเวลา

เส้นทางของทุกข่าวใน train ถูกรวมเป็น **Social Heterogeneous Graph หนึ่งกราฟต่อ dataset** ซึ่งเป็น input ของ Phase I ส่วนรายละเอียดของข้อมูลผมจะกลับมาอีกครั้งในหัวข้อ Data

▸ พูดสั้น ๆ เป็นภาพรวม รายละเอียดอยู่สไลด์ 22



9 Methodology · 2 Phase 3 Module

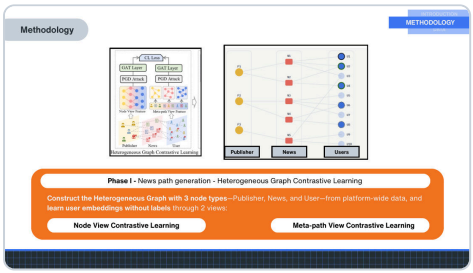
~40 วินาที

เพื่อให้เข้าใจง่าย ผมแบ่งเป็น 2 Phase ครับ

Phase I – News path generation สร้างเส้นทางการแพร่ ประกอบด้วย Heterogeneous Graph Contrastive Learning และ Reinforced Propagation Path Generation

Phase II – Classifier ตัดสินว่าข่าว Fake หรือ Real ด้วย Temporal-aware Fake News Detection

▸ ชี้แถบส้ม = Phase I, แถบน้ำเงิน = Phase II

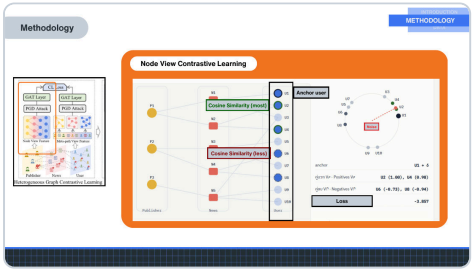


10 Phase I · Heterogeneous Graph Contrastive Learning

~50 วินาที

Module แรก เริ่มจากสร้าง **Heterogeneous Graph** ที่มี node 3 ประเภท คือ Publisher สีเหลือง, News สีแดง และ User สีน้ำเงิน จากข้อมูลทั้งแพลตฟอร์ม

เป้าหมายคือเรียนรู้ **user embedding** หรือเวกเตอร์แทนตัวตนของผู้ใช้แต่ละคน **โดยไม่ใช้ label** ผ่าน 2 มุมมอง คือ Node View และ Meta-path View



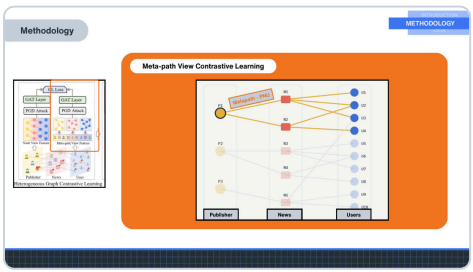
11 Node View Contrastive Learning

~60 วินาที

มุมมองแรกคือ **Node View** เลือกผู้ใช้หนึ่งคนเป็น **anchor** ในตัวอย่างคือ U1 แล้วหาผู้ใช้ที่คล้ายที่สุดเป็น **positive** เช่น U2 และ U4 ที่ cosine similarity สูง และผู้ใช้ที่ต่างที่สุดเป็น **negative** เช่น U6 และ U8

นอกจากนี้ยังใส่ **noise** ด้วย PGD Attack ให้ embedding ทนต่อการรบกวน แล้วคำนวณ **contrastive loss** ที่ดึง positive เข้าใกล้และผลัก negative ออกไป

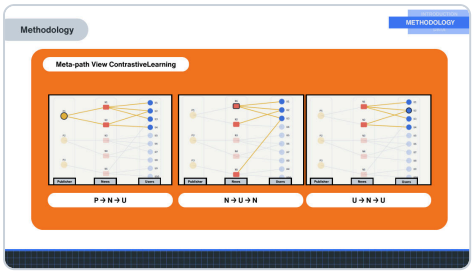
▸ ชี้ anchor → คู่บวก → คู่ลบ → ค่า loss



12 Meta-path View · PNU

~40 วินาที

มุมมองที่สองคือ **Meta-path View** ใช้เส้นทางที่มีความหมายในกราฟ ตัวอย่างนี้คือ meta-path **P → N → U** คือ Publisher โพสต์ข่าว แล้ว User แชร์ข่าวนั้น ผู้ใช้ที่เชื่อมผ่าน publisher เดียวกันจะถือว่ามีความสัมพันธ์กัน

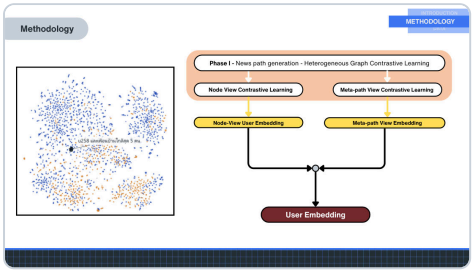


13 Meta-path View · 3 แบบ

~45 วินาที

paper ใช้ meta-path 3 แบบ คือ **P→N→U** ผู้ใช้ที่แชร์ข่าวจากสำนักข่าวเดียวกัน, **N→U→N** ข่าวที่มีผู้ใช้คนเดียวกันแชร์ และ **U→N→U** ผู้ใช้ที่แชร์ข่าวเดียวกัน

แต่ละแบบให้ embedding คนละมุม แล้วนำมารวมกันเป็น meta-path view embedding



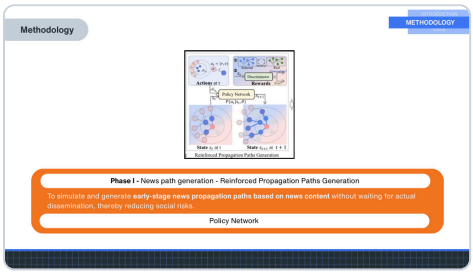
14 รวมเป็น User Embedding (ตัวอย่างจริง)

~55 วินาที

สอง view นี้รวมกันเป็น **User Embedding** ภาพซ้ายคือผลจริงจากที่เรารัน PHEME ลดมิติด้วย t-SNE แต่ละจุดคือผู้ใช้หนึ่งคน สีน้ำเงินคือคนที่เคยแชร์แต่ข่าวจริง สีส้มคือคนที่แชร์ข่าวปลอม

จะเห็นว่าผู้ใช้แยกเป็นกลุ่ม และมีกลุ่มที่เป็นสีส้มหนาแน่น ทั้งที่ module นี้ไม่ได้ใช้ label เลย ตัวอย่างผู้ใช้ u258 อยู่ใกล้เพื่อนบ้าน 5 คนที่เคย repost ต่อกันจริง

- เน้นว่า “ไม่ได้ใช้ label” แต่ยังแยกกลุ่มได้

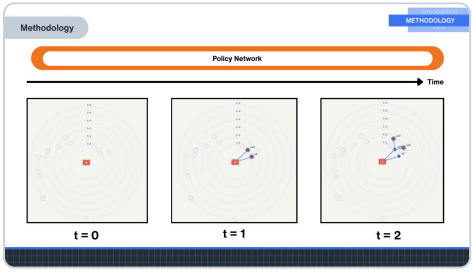


15 Phase I · Reinforced Propagation Paths Generation

~40 วินาที

Module ที่สองคือ **Reinforced Propagation Paths Generation** มีหน้าที่จำลองและสร้างเส้นทางการแพร่ช่วงแรกจากเนื้อหาข่าว โดยไม่ต้องรอการแพร่จริง เพื่อลดความเสี่ยงทางสังคม

หัวใจคือ **Policy Network** ซึ่งเป็น agent แบบ reinforcement learning



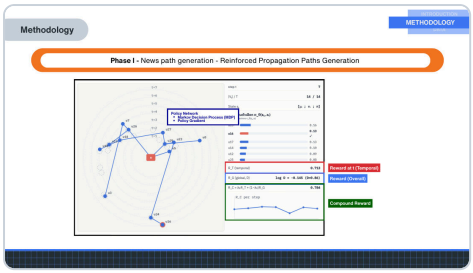
16 Policy Network ทีละ step

~55 วินาที

ภาพนี้แสดงการทำงานทีละ step ครับ ที่ **t = 0** มีแค่ข่าวสีแดงตรงกลาง ที่ **t = 1** policy เลือกผู้ใช้กลุ่มแรกมาต่อจากข่าว ที่ **t = 2** เลือกผู้ใช้เพิ่มต่อจากคนที่อยู่ในเส้นทางแล้ว

ในแต่ละ step policy จะเลือกผู้ใช้ K2 = 10 คน ทำซ้ำจนครบ T = 200 คน ได้เป็นเส้นทาง G′

- ชี้ภาพทีละช่องจากซ้ายไปขวา



17 MDP และ Reward

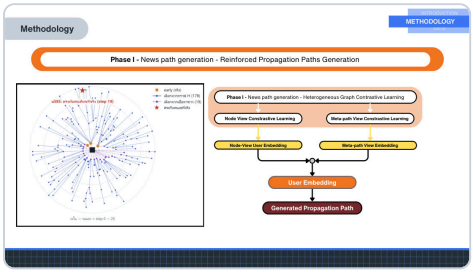
~70 วินาที

ปัญหานี้ถูกมองเป็น **Markov Decision Process** state คือเส้นทางปัจจุบันกับเนื้อหาข่าว action คือการเลือกผู้ใช้คนถัดไป และฝึก policy ด้วย policy gradient แบบ actor-critic

reward มี 2 ส่วน **Reward at t (Temporal)** ให้คะแนนเมื่อผู้ใช้ที่เลือกคล้ายกับคนที่แชร์จริงในช่วงเวลานั้น และ **Reward (Overall)** จาก discriminator ที่ตัดสินว่าเส้นทางทั้งเส้นดูเหมือนของจริงแค่ไหน

สองส่วนรวมเป็น **Compound Reward** ในตัวอย่างจริงนี้ R_T = 0.713 และ R_C = 0.784 ครับ

▸ ชี้กล่องแดง (temporal) → น้ำเงิน (overall) → เขียว (compound)



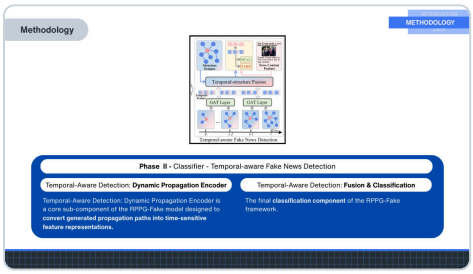
18 ผลลัพธ์ Phase I · Generated Propagation Path

~55 วินาที

นี่คือเส้นทางที่ model สร้างจริงจากข่าว PHEME หนึ่งข่าว เริ่มจากผู้ใช้จริง 3 คน ตรงกลาง แล้วสร้างเพิ่มเป็น **197 คนใน 20 step** ส่วนใหญ่ 178 คนเลือกผ่านกราฟ H อีก 19 คนเลือกจากความใกล้กับเนื้อหาข่าว

ที่น่าสังเกตคือมีแค่ **u585** ดาวสีแดงคนเดียวที่ตรงกับคนที่แชร์จริง ซึ่งเป็นจุดที่เราจะกลับมาพูดในช่วงผลการทดลอง

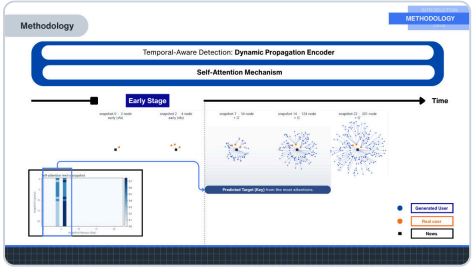
สรุป Phase I: Contrastive Learning ให้ User Embedding แล้ว RL ใช้ embedding นั้นสร้าง Generated Propagation Path



19 Phase II · Temporal-aware Fake News Detection

~40 วินาที

Phase II มี 2 ส่วน ส่วนแรก **Dynamic Propagation Encoder** แปลงเส้นทางที่สร้างขึ้นให้เป็น feature ที่รู้มิติเวลา ส่วนที่สอง **Fusion & Classification** รวมทุกอย่างแล้วตัดสินว่าข่าวจริงหรือปลอม

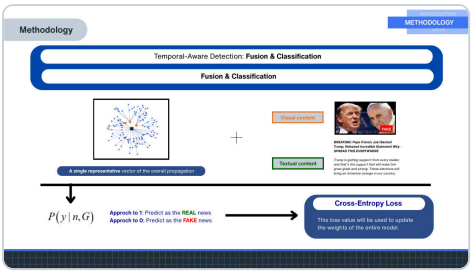


20 Dynamic Propagation Encoder

~60 วินาที

เส้นทางถูกตัดเป็น **snapshot** ตามเวลา ช่วงแรกคือ **early stage** ที่มีแค่ผู้ใช้จริงจุดสีส้ม จากนั้นผู้ใช้ที่สร้างขึ้นจุดสีน้ำเงินค่อย ๆ เพิ่มเข้ามา จาก 2 node จนถึง 201 node

แต่ละ snapshot ผ่าน GAT ได้เวกเตอร์หนึ่งตัว แล้วใช้ **Self-Attention** หาว่า snapshot ไหนสำคัญ ตาราง heatmap ด้านล่างซ้ายคือค่า attention จริงจาก model ของเรา



21 Fusion & Classification

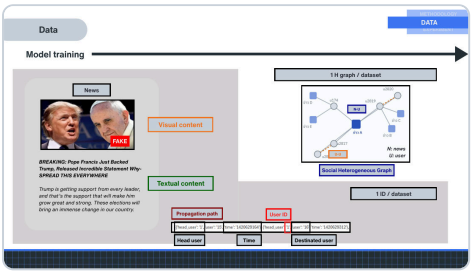
~60 วินาที

ขั้นสุดท้าย เส้นทางทั้งหมดถูกรวมเป็น **เวกเตอร์ตัวแทนเพียงตัวเดียว** แล้วต่อกับ feature ของ **ภาพ** และ **ข้อความ** ของข่าว ตัวอย่างคือข่าวปลอมที่อ้างว่าพระสันตะปาปาสนับสนุนทรัมป์

ผลออกมาเป็นความน่าจะเป็น **P(y | n, G)** ถ้าเข้าใกล้ 1 คือข่าวจริง ถ้าเข้าใกล้ 0 คือข่าวปลอม แล้วใช้ **Cross-Entropy Loss** ปรับน้ำหนักของทั้ง model

---

Data · 3:05 นาที

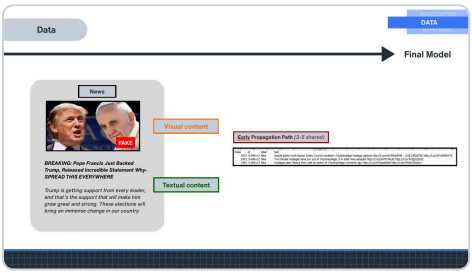


22 Data · ตอน train

~60 วินาที

ในส่วนของข้อมูล ตอน train ข่าวแต่ละข่าวมี 3 ส่วน คือ **ภาพ**, **ข้อความ** และ **เส้นทางการแพร่** ซึ่งเก็บเป็นรายการของ head user คือคนที่ถูกแชร์ต่อ, user ID ของคนที่แชร์ และเวลา

เส้นทางของทุกข่าวใน train ถูกรวมเป็น **Social Heterogeneous Graph เพียงกราฟเดียวต่อ dataset** โดยมี edge แบบ N-U คือผู้ใช้แชร์ข่าว และ U-U คือผู้ใช้แชร์ต่อจากกัน และ user ID หนึ่งชุดต่อ dataset

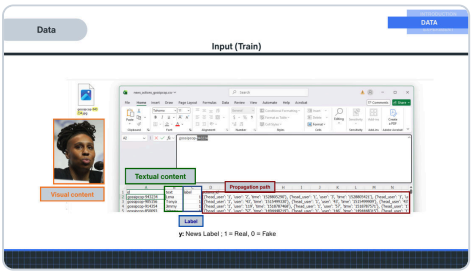


23 Data · ตอนใช้งานจริง (Final Model)

~40 วินาที

ตอนใช้งานจริง model ไม่ต้องรู้เส้นทางทั้งหมด ต้องการแค่ภาพ ข้อความ และ **early propagation path** คือผู้ใช้ 3–5 คนแรกที่แชร์ แล้ว model จะสร้างเส้นทางที่เหลือเอง

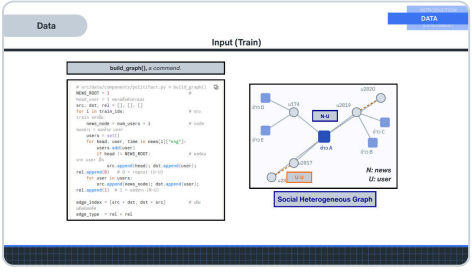
▸ เทียบกับหน้าก่อน: train ใช้เส้นทางเต็ม, ใช้งานใช้แค่ 3–5 คนแรก



24 Data · หน้าตา input จริง

~40 วินาที

นี่คือไฟล์ข้อมูลจริงที่เราใช้ ตัวอย่าง GossipCop แต่ละแถวคือข่าวหนึ่งข่าว มีคอลัมน์ข้อความ, label และ propagation path ส่วนภาพเก็บเป็นไฟล์แยก ตั้งชื่อตาม id ของข่าว เพื่อจับคู่กับข้อความ



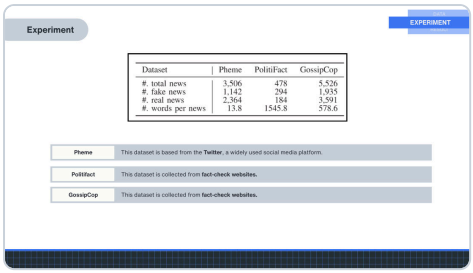
25 Data · สร้างกราฟด้วย build_graph()

~45 วินาที

กราฟ H สร้างด้วยฟังก์ชัน **build_graph()** ในไฟล์ politifact.py วนทุกข่าวใน train สร้าง edge จากผู้ใช้ไปข่าว (N-U) และถ้าผู้ใช้แชร์ต่อจากผู้ใช้อื่น ก็สร้าง edge แบบ U-U

สำคัญคือ **ใช้เฉพาะข่าว train** เพื่อไม่ให้ข้อมูล test รั่วเข้าไปในกราฟ

Experiment · 2:30 นาที

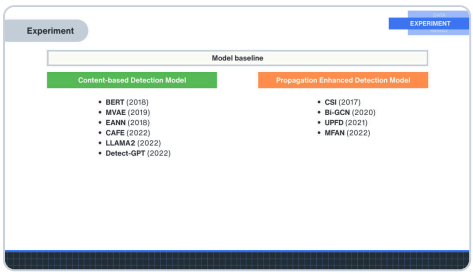


26 Experiment · Dataset

~45 วินาที

paper ทดลองบน 3 dataset คือ **PHEME** จาก Twitter 3,506 ข่าว, **PolitiFact** จากเว็บ fact-check 478 ข่าว และ **GossipCop** ข่าวบันเทิง 5,526 ข่าว

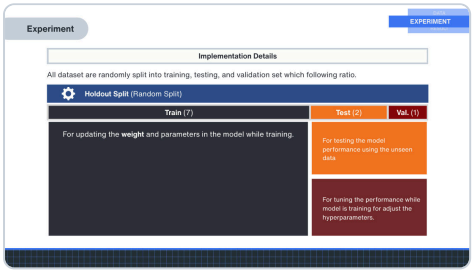
สังเกตว่า PolitiFact มีข่าวน้อยมากแต่ข้อความยาวที่สุด เฉลี่ยกว่า 1,500 คำต่อข่าว



27 Experiment · Baseline

~35 วินาที

paper เทียบกับ baseline 10 ตัว แบ่งเป็น **Content-based** 6 ตัว เช่น BERT, MVAE, CAFE รวมถึง LLAMA2 และ Detect-GPT และ **Propagation Enhanced** 4 ตัว คือ CSI, Bi-GCN, UPFD และ MFAN

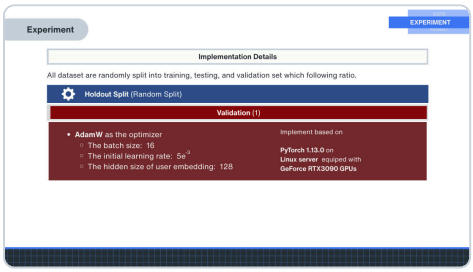


28 Experiment · Holdout split

~40 วินาที

ข้อมูลถูกแบ่งแบบสุ่มเป็น train 70%, validation 10% และ test 20% train ใช้ปรับน้ำหนัก validation ใช้เลือกค่า hyperparameter และ test ใช้วัดผลกับข้อมูลที่ไม่เคยเห็น

ข้อสังเกตคือเป็นการแบ่งครั้งเดียว ไม่ได้ทำ cross validation ซึ่งผมจะกลับมาพูดในส่วนวิพากษ์

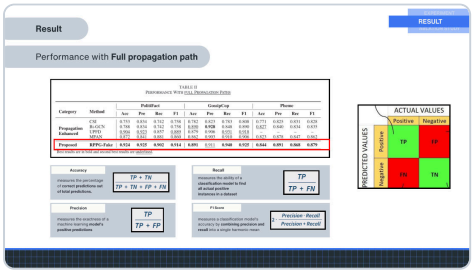


29 Experiment · Implementation

~30 วินาที

ค่าที่ paper ระบุคือ optimizer **AdamW**, batch size 16, learning rate 5×10^{-3} และ user embedding ขนาด 128 รันบน PyTorch 1.13 และ GPU RTX 3090

Result · 2:35 นาที



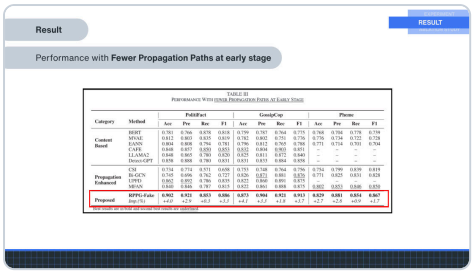
30 Result · Full propagation path

~70 วินาที

ก่อนดูผล ทบทวน metric 4 ตัว **Accuracy** คือสัดส่วนที่ทายถูกทั้งหมด **Precision** คือในที่ทายว่าเป็นบวก ถูกกี่ส่วน **Recall** คือในของจริงที่เป็นบวก หาเจอกี่ส่วน และ **F1** คือค่าเฉลี่ยฮาร์มอนิกของ precision และ recall

Table II เป็นกรณีที่มีเส้นทางเต็ม RPPG-Fake ได้ accuracy **0.924** บน PolitiFact, **0.891** บน GossipCop และ **0.844** บน PHEME ดีที่สุดเกือบทุกช่อง

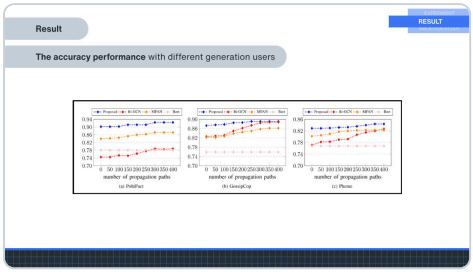
▸ ชี้แถวกรอบแดงในตาราง



31 Result · Early stage

~50 วินาที

Table III คือกรณีสำคัญที่สุด คือ **early stage** ที่มีผู้แชร์แค่ 3–5 คน RPPG-Fake ได้ **0.902, 0.873 และ 0.829** ดีกว่า baseline ที่ดีที่สุด **2.7 ถึง 4.1 pp** ซึ่งเป็นตัวเลขหลักที่ paper ใช้ยืนยันว่าแนวคิดนี้ได้ผล

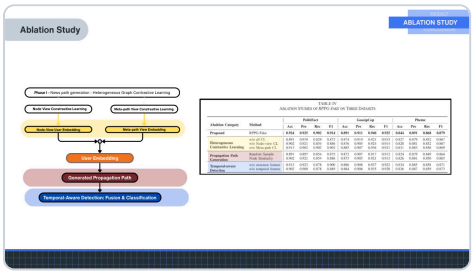


32 Result · จำนวนผู้ใช้ที่สร้าง

~35 วินาที

กราฟนี้แสดงว่าเมื่อเพิ่มจำนวนผู้ใช้ที่สร้างในเส้นทาง accuracy ของ RPPG-Fake เส้นสีน้ำเงินสูงกว่า baseline อื่นตลอด และดีขึ้นเล็กน้อยเมื่อสร้างมากขึ้น

Ablation Study · 0:55 นาที



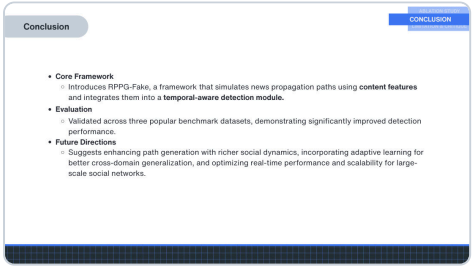
33 Ablation Study

~55 วินาที

Table IV ตัดหรือเปลี่ยนแต่ละส่วนของ model ผลลดลงทุกกรณี ตั้งแต่ 0.5 ถึง 3.3 pp ที่ลดมากที่สุดคือการตัด contrastive learning ทั้งหมด และการเปลี่ยน RL เป็นการสุ่มผู้ใช้ (Random Sample)

paper ใช้ตารางนี้ยืนยันว่าทุก module มีประโยชน์ แต่ขอให้สังเกตว่า **ไม่มีแถวที่ไม่ใช้ G′ เลย** ซึ่งเราทดลองเพิ่มเอง

Conclusion · 0:45 นาที



34 Conclusion (ของ paper)

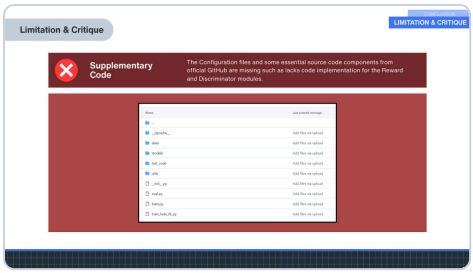
~45 วินาที

สรุปของผู้เขียน RPPG-Fake จำลองเส้นทางการแพร่จากเนื้อหาข่าวแล้วรวมเข้ากับ temporal-aware detection ทดสอบบน 3 dataset และดีกว่า baseline

งานในอนาคตที่ผู้เขียนเสนอคือ สร้างเส้นทางที่คำนึงถึงพลวัตทางสังคมมากขึ้น ปรับตัวข้ามโดเมน และทำให้ใช้งานแบบ real time กับเครือข่ายขนาดใหญ่ได้

▸ จบส่วนของ paper → เข้าสู่สิ่งที่เราทำและวิพากษ์

Limitation & Critique · 5:40 นาที

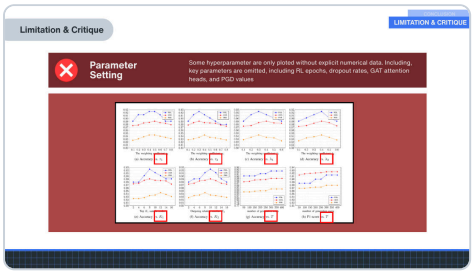


35 Critique · Supplementary Code

~50 วินาที

ส่วนต่อไปคือการวิพากษ์จากการที่เราลองทำซ้ำจริงครับ ข้อแรก **code ที่เผยแพร่ใช้ไม่ได้** ไฟล์ config และ source code บางส่วนหายไป โดยเฉพาะ **ส่วน Reward และ Discriminator** ที่เป็นหัวใจของ Module 2 ไม่มี implementation

นอกจากนี้ยังพบตัวแปรที่ไม่ได้ประกาศ และ validation กับ test ถูกสลับกัน เราจึงต้องเขียนทั้ง 3 module ใหม่ตามสมการใน paper



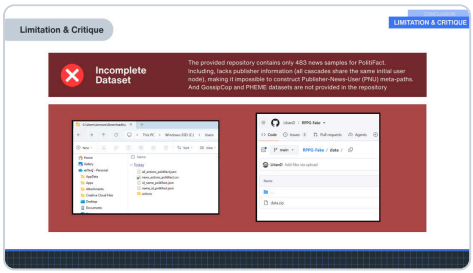
36 Critique · Parameter Setting

~45 วินาที

ข้อสอง **ค่าพารามิเตอร์ไม่ครบ** ค่าสำคัญอย่าง τ1, τ2, λ1, λ2, K1, K2 และ T มีให้แค่เป็นกราฟใน Fig. 4 ไม่มีตัวเลขที่ใช้จริง และไม่ระบุจำนวน epoch ของ RL, dropout, จำนวน attention head และค่า PGD

เราจึงเลือกค่าจากจุดสูงสุดของแต่ละกราฟ

▸ ชี้กรอบแดงในกราฟ Fig. 4

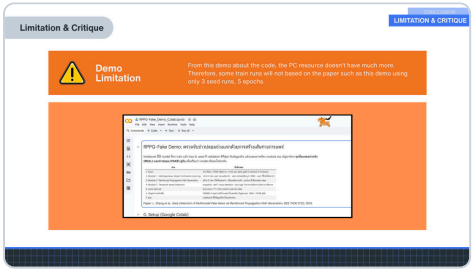


37 Critique · Incomplete Dataset

~45 วินาที

ข้อสาม **ข้อมูลไม่ครบ** repository มีแค่ PolitiFact 483 ข่าว ไม่มีภาพ ไม่มีข้อมูล publisher เพราะทุก cascade เริ่มจาก user เดียวกัน ทำให้สร้าง meta-path PNU ไม่ได้ และไม่มี GossipCop กับ PHEME ให้

เราจึงสร้าง GossipCop และ PHEME ขึ้นใหม่เอง โดยดึงเนื้อข่าวและภาพจากแหล่งต้นทาง และใช้ meta-path UU แทน PNU



38 Limitation ของเรา · Demo

~45 วินาที

ข้อจำกัดของเราเอง เครื่องที่ใช้เป็น laptop GPU 8 GB และ RAM 16 GB การ train ใหม่ทั้งชุดใช้เวลามากกว่า 6 ชั่วโมง จึงต้องลดเหลือ **3 seeds** จาก 5 ใน paper และ **RL 5 epoch**

เราทำ notebook บน Google Colab ไว้ ให้รัน model ที่ train แล้ว และดูภาพของทุก module ได้ทันที โดยแสดงข่าวจริงกับข่าวปลอมคู่กัน

▸ ถ้ามีเวลา เปิด Colab โชว์สั้น ๆ



39 Paper vs ของเรา

~55 วินาที

ตารางนี้สรุปว่าเราทำต่างจาก paper ตรงไหน ด้าน **dataset** เราใช้เฉพาะข่าวที่มีครบ ข้อความ ภาพ และเส้นทาง PHEME จึงเหลือ 2,004 ข่าว GossipCop 2,827 ข่าว และ PolitiFact ไม่มีภาพ

ด้าน **parameter** เราใช้ CLIP เป็น encoder, contrastive 300 step, RL 5 epoch, detector สูงสุด 30 epoch และแก้ bug เพิ่ม 3 จุด ส่วนที่เหมือน paper คือ AdamW, learning rate, batch size, hidden size, การแบ่ง 7:1:2 และนิยาม early stage

- ชี้ป้าย Incomplete Dataset และ Parameter Setting



40 ผลของเราเทียบ paper

~75 วินาที

ผลของเรารอบสุดท้ายเฉลี่ย 3 seeds **PHEME** ได้ early accuracy 0.837 เทียบ paper 0.829 และ **GossipCop** 0.878 เทียบ 0.873 คือ **ตรงกับ paper** และ full path ก็ดีกว่า early ทุก dataset เหมือน paper

ที่ยังไม่ตรงคือ **PolitiFact ต่ำกว่า 7.6 pp** ข้อมูลไม่มีภาพ และ test มีแค่ 96 ข่าว

ข้อค้นพบสำคัญคือคอลัมน์สุดท้าย ถ้า **ไม่ใช้ G′ เลย** ได้ 0.868, 0.900 และ 0.844 ซึ่ง **เท่ากันหรือดีกว่า** การใช้ G′ ทุก dataset ข้ออ้างหลักของ paper ว่าเส้นทางที่สร้างช่วยตรวจจับ จึงยังยืนยันไม่ได้จากการทดลองของเรา

- เน้นคอลัมน์ Δ และคอลัมน์ “ไม่ใช้ G′”



41 Source Code

~25 วินาที

สุดท้ายคือแหล่ง code ทั้งหมด Google Colab สำหรับ demo, GitHub ของเราที่มี code ซึ่งแก้ให้รันได้ พร้อมผลการทดลองทั้งหมด และ GitHub ทางการของผู้เขียน เพื่อเปรียบเทียบ

Closing · 0:15 นาที



42 Thank you

~15 วินาที

ขอบคุณครับ ยินดีรับคำถามและข้อเสนอแนะครับ

เตรียมตอบคำถาม

ถาม: ทำไม PolitiFact ไม่ตรงกับ paper

มี 3 สาเหตุ (1) ข้อมูลของผู้เขียนไม่มีภาพ และ test มีแค่ 96 ข่าว 1 ข่าว ≈ 1 pp (2) ตัวเลข PolitiFact ใน paper หารด้วย 92 ลงตัวทุกค่า แปลว่าวัดบนข่าวชุดเดียว 92 ข่าวครั้งเดียว ไม่ได้เฉลี่ย 5 รอบ และ code ทางการใช้ชุดนี้เลือก checkpoint ด้วย เมื่อเราเลือก epoch บนชุดเดียวกับที่รายงานแบบนั้น ผลขึ้นจาก 0.826 เป็น 0.878 (3) logistic regression บนข้อความอย่างเดียวได้ 0.892 แสดงว่า detector ของเรายังเรียนจากข้อมูลน้อย ๆ ได้ไม่ดี

ถาม: G′ ช่วยจริงไหม

จากการทดลองของเรา ไม่ช่วย ไม่ใช้ G′ ได้เท่ากันหรือดีกว่าทุก dataset และผู้ใช้ที่ model สร้างตรงกับคนที่แชร์จริงแค่ราว 0.1% (PHEME) ถึง 5% (GossipCop) ใน PHEME detector แทบไม่ใช้ส่วนของเส้นทาง ผลมาจากข้อความกับภาพ

ถาม: มี data leakage ไหม

ตรวจแล้วไม่พบ กราฟ H สร้างจากข่าว train เท่านั้น และเมื่อสลับ label ของ train แบบสุ่ม accuracy ตกลงเหลือระดับ majority class

ถาม: ทำ cross validation ไหม

paper ไม่ได้ทำ ใช้ hold-out 7:1:2 ครั้งเดียว ไม่รายงาน std ของเราสุ่ม split ใหม่ทุก seed และรายงาน ± std แต่ยังไม่ใช่ k-fold

ถาม: ใช้กับข่าวใหม่ได้ไหม

ได้ไม่ดี model ต้องรู้จักผู้ใช้ล่วงหน้า ข่าวจาก PHEME 4 เหตุการณ์ที่ไม่อยู่ใน training ได้ accuracy 0.25 (AUC 0.75) เพราะผู้ใช้ส่วนใหญ่เป็นคนใหม่

ถาม: ทำไมใช้แค่ 3 seeds

ข้อจำกัดเครื่อง laptop GPU 8 GB และ RAM 16 GB การ train ใหม่ทั้งชุดใช้มากกว่า 6 ชั่วโมง จึงลดเป็น 3 seeds และ RL 5 epoch

จุดที่ควรแก้บนสไลด์ก่อนนำเสนอ

หน้า	รายละเอียด
7	y: News Label เขียน 2 = Fake แต่ใน paper คือ 0 = Fake
11	หัวข้อ “Constrastive” สะกดผิด (ควรเป็น Contrastive) ใช้ซ้ำในหน้า 10–14, 18, 33
28	“For juning the performance” ควรเป็น “For tuning”
37	ประโยคขึ้นต้นด้วย “he provided repository” ตัว T หาย
38	“5 epoches” ควรเป็น “5 epochs”